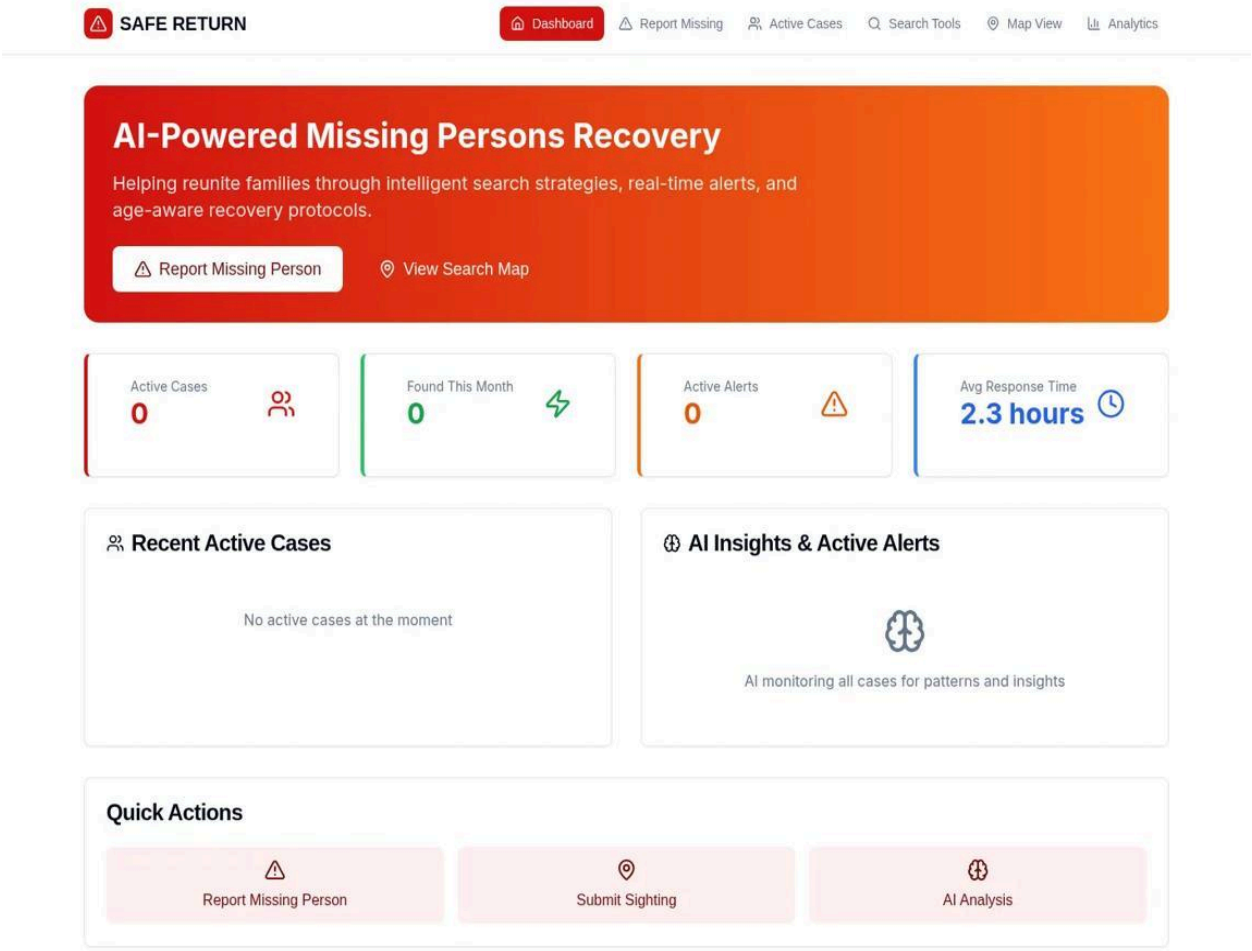
FindBack — Cloud Computing Project Plan



Project title: FindBack: A Cloud-Based Missing Person and Community Alert Network

Project type: Full-stack web application with cloud storage, location-based alerts, and community reporting.

Recommended scope: Balanced — substantial enough for a university cloud computing project, but manageable for a student team.

1. Project overview

FindBack will help families and authorized reporters publish missing-person cases, allow community members to view approved alerts, and let people submit possible sightings securely.

The key idea is to connect people through a centralized, cloud-hosted system that can distribute relevant alerts across different locations.

Problem statement

When a person goes missing, information can become scattered across social media posts, messaging groups, and local communities. Information may be outdated, difficult to verify, or unable to reach people in nearby areas.

FindBack aims to provide a centralized platform for managing missing-person reports, moderating public alerts, collecting sightings, and notifying users who choose to receive alerts for selected areas.

Main objectives

- Provide a structured way to submit and manage missing-person reports.
- Help community members discover approved cases by location.
- Allow sightings to be submitted privately for review.
- Notify subscribed users when relevant cases are approved.
- Store case records and images using cloud services.
- Demonstrate cloud computing through managed services, asynchronous notifications, monitoring, and load testing.

Important scope boundary: FindBack is a community assistance platform, not an official missing-person database or emergency response service. Reports must be reviewed, and urgent cases should still be reported to the appropriate authorities.

2. Functional requirements

These are the features the application should actually implement.

Public and registered users

Account management

Register, log in, log out, and manage basic profile information. Users can subscribe to selected alert areas.

Missing-person directory

Browse approved cases, search by name or case reference, filter by area and status, and view case details.

Case submission

Submit a description, photograph, last-seen area, approximate date, and safe contact method. New submissions remain private until approved.

Map and area discovery

Display approved cases on a map using approximate locations. Users can filter cases by district, city, or a selected area.

Sightings

Submit a possible sighting with the time, approximate location, and optional supporting information. Sightings are visible only to authorized reviewers and the relevant case team.

Area-based alerts

Notify subscribed users when a moderator approves a case in an area they selected. Include a link to the approved public case.

Moderator dashboard

Review reports, approve or reject submissions, flag duplicates, inspect sightings, handle abuse reports, and resolve cases.

Case statuses

Use a clear lifecycle:

`Pending review → Approved → Resolved`

A separate `Rejected` status handles reports that should not be published. An approved case can also be temporarily hidden or flagged for further review without deleting its audit history.

Do not treat a report as officially verified merely because a moderator has approved it for publication. Make the distinction clear in the interface.

3. Non-functional requirements

These requirements define how reliably and safely the system should work.

Requirement	Practical target

Security	Enforce authentication, role-based permissions, and private sighting reports
Performance	Aim for common API requests to respond within 1–2 seconds under your test load
Availability	Deploy the application to a cloud environment and monitor errors and downtime
Scalability	Process notifications asynchronously instead of making users wait for every email
Usability	Responsive design for mobile phones and desktops
Reliability	Handle failed notifications, retry safely, and prevent duplicate alerts
Privacy	Minimize personal data, restrict access, and provide a case-removal process
Maintainability	Use documented APIs, automated tests, and Git branches

The performance numbers are project targets to measure, not guarantees. Record actual latency and error rates during testing.

4. System architecture

I recommend a simple three-layer application with one background notification worker. This gives you meaningful cloud architecture without introducing unnecessary microservices.

Presentation layer

React + Vite · Responsive interface · Leaflet map

Application layer

FastAPI REST API

Authentication · Permissions · Reports · Sightings · Moderation

PostgreSQL

Persistent records and subscriptions

Object storage

Case photographs and attachments

Asynchronous notification pipeline

Queue → background worker or cloud function → email provider

The API handles requests; the worker handles notification delivery separately.

Example: what happens when a report is approved?

1. A moderator approves a pending case through the React dashboard.
2. FastAPI validates the moderator's permissions and updates PostgreSQL.
3. The backend creates a notification job for that case.
4. A cloud queue stores the job until a worker can process it.
5. The worker finds matching area subscriptions and sends emails.
6. Delivery attempts and failures are logged, and failed jobs can be retried.

This is a useful demonstration of event-driven cloud computing. For example, AWS provides a documented pattern combining SQS, Lambda, a relational database, and CloudWatch monitoring.



Amazon Relational Database Service

5. Database design

Use PostgreSQL as the primary database. It is a good fit because the system has related records: users submit cases, cases receive sightings, moderators review reports, and users subscribe to areas.

The following schema is a practical starting point.

1. `users`

Stores account and access information.

Column	Type	Purpose
`id`	UUID, PK	User identifier
`email`	VARCHAR, UNIQUE	Login and notification email
`password_hash`	TEXT	Secure password hash

`display_name`	VARCHAR	Public display name
`role`	ENUM	`user`, `moderator`, `admin`
`created_at`	TIMESTAMPTZ	Account creation
`is_active`	BOOLEAN	Account status

2. `missing_person_cases`

Main case records.

Column	Type	Purpose
`id`	UUID, PK	Internal case identifier
`case_code`	VARCHAR, UNIQUE	Public reference code
`submitted_by`	UUID, FK	Reporter
`person_name`	VARCHAR	Name, if appropriate to publish
`description`	TEXT	Physical description and context
`last_seen_at`	TIMESTAMPTZ	Last known date and time
`area_name`	VARCHAR	Publicly displayed area
`latitude` / `longitude`	DECIMAL, nullable	Restricted or approximate location
`photo_key`	TEXT, nullable	Object-storage key, not a public secret
`status`	ENUM	`pending`, `approved`, `rejected`, `resolved`
`reviewed_by`	UUID, FK, nullable	Moderator
`created_at`	TIMESTAMPTZ	Submission time

3. `sightings`

Private submissions associated with a case.

Column	Type	Purpose
`id`	UUID, PK	Sighting identifier
`case_id`	UUID, FK	Related case
`submitted_by`	UUID, FK, nullable	Reporter, if registered
`observed_at`	TIMESTAMPTZ	When the sighting occurred
`location_description`	TEXT	Approximate location
`notes`	TEXT	Submitted observations
`status`	ENUM	`pending`, `reviewed`, `dismissed`
`created_at`	TIMESTAMPTZ	Submission time

4. `area_subscriptions`

Maps users to areas they want to receive alerts for.

Column	Type	Purpose
`id`	UUID, PK	Subscription identifier
`user_id`	UUID, FK	Subscriber
`area_code`	VARCHAR	District or defined area
`created_at`	TIMESTAMPTZ	Subscription time

Add a unique constraint on `(user_id, area_code)` to prevent duplicate subscriptions.

5. `notification_jobs`

Tracks notification delivery and retries.

Column	Type	Purpose
`id`	UUID, PK	Job identifier
`case_id`	UUID, FK	Related case
`user_id`	UUID, FK	Intended recipient
`channel`	VARCHAR	Email or push
`status`	ENUM	`pending`, `sent`, `failed`
`attempts`	INTEGER	Retry count
`sent_at`	TIMESTAMPTZ, nullable	Successful delivery time

6. `audit_logs`

Records sensitive administrative actions.

Column	Type	Purpose
`id`	UUID, PK	Log identifier
`actor_id`	UUID, FK	User performing the action
`case_id`	UUID, FK, nullable	Related case
`action`	VARCHAR	Approved, rejected, resolved, etc.
`created_at`	TIMESTAMPTZ	Action time

Relationships

#chatgpt-mermaid-_r_tl_{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,Segoe UI,Helvetica,Apple Color Emoji,Arial,sans-serif,Segoe UI Emoji,Segoe UI

Symbol;font-size:16px;fill:rgb(237, 237, 237);}@keyframes
edge-animation-frame{from{stroke-dashoffset:0;}}@keyframes
dash{to{stroke-dashoffset:0;}}#chatgpt-mermaid-_r_tl_
.edge-animation-slow{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 50s
linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_tl_
.edge-animation-fast{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 20s
linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_tl_ .error-icon{fill:rgb(48, 48,
48);}#chatgpt-mermaid-_r_tl_ .error-text{fill:rgb(237, 237, 237);stroke:rgb(237, 237,
237);}#chatgpt-mermaid-_r_tl_
.edge-thickness-normal{stroke-width:1px;}#chatgpt-mermaid-_r_tl_
.edge-thickness-thick{stroke-width:3.5px;}#chatgpt-mermaid-_r_tl_
.edge-pattern-solid{stroke-dasharray:0;}#chatgpt-mermaid-_r_tl_
.edge-thickness-invisible{stroke-width:0;fill:none;}#chatgpt-mermaid-_r_tl_
.edge-pattern-dashed{stroke-dasharray:3;}#chatgpt-mermaid-_r_tl_
.edge-pattern-dotted{stroke-dasharray:2;}#chatgpt-mermaid-_r_tl_ .marker{fill:rgb(175, 175,
175);stroke:rgb(175, 175, 175);}#chatgpt-mermaid-_r_tl_ .marker.cross{stroke:rgb(175, 175,
175);}#chatgpt-mermaid-_r_tl_
svg{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,Segoe
UI,Helvetica,Apple Color Emoji,Arial,sans-serif,Segoe UI Emoji,Segoe UI
Symbol;font-size:16px;}#chatgpt-mermaid-_r_tl_ p{margin:0;}#chatgpt-mermaid-_r_tl_
.entityBox{fill:rgb(9, 23, 44);stroke:rgb(31, 78, 148);}#chatgpt-mermaid-_r_tl_
.relationshipLabelBox{fill:rgb(48, 48, 48);opacity:0.7;background-color:rgb(48, 48,
48);}#chatgpt-mermaid-_r_tl_ .relationshipLabelBox rect{opacity:0.5;}#chatgpt-mermaid-_r_tl_
.labelBkg{background-color:rgba(48, 48, 48, 0.5);}#chatgpt-mermaid-_r_tl_
.edgeLabel{background-color:rgb(0, 0, 0);}#chatgpt-mermaid-_r_tl_ .edgeLabel .label
rect{fill:rgb(0, 0, 0);}#chatgpt-mermaid-_r_tl_ .edgeLabel .label text{fill:rgb(237, 237,
237);}#chatgpt-mermaid-_r_tl_ .edgeLabel .label{fill:rgb(31, 78,
148);font-size:14px;}#chatgpt-mermaid-_r_tl_
.label{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,Segoe
UI,Helvetica,Apple Color Emoji,Arial,sans-serif,Segoe UI Emoji,Segoe UI Symbol;color:rgb(237,
237, 237);}#chatgpt-mermaid-_r_tl_
.edge-pattern-dashed{stroke-dasharray:8,8;}#chatgpt-mermaid-_r_tl_ .node
rect,#chatgpt-mermaid-_r_tl_ .node circle,#chatgpt-mermaid-_r_tl_ .node
ellipse,#chatgpt-mermaid-_r_tl_ .node polygon{fill:rgb(9, 23, 44);stroke:rgb(31, 78,
148);stroke-width:1px;}#chatgpt-mermaid-_r_tl_ .relationshipLine{stroke:rgb(175, 175,
175);stroke-width:1px;fill:none;}#chatgpt-mermaid-_r_tl_
.marker{fill:none!important;stroke:rgb(175, 175,
175)!important;stroke-width:1;}#chatgpt-mermaid-_r_tl_
[data-look=neo].labelBkg{background-color:rgba(48, 48, 48, 0.5);}#chatgpt-mermaid-_r_tl_
.node .neo-node{stroke:rgb(31, 78, 148);}#chatgpt-mermaid-_r_tl_ [data-look="neo"].node
rect,#chatgpt-mermaid-_r_tl_ [data-look="neo"].cluster rect,#chatgpt-mermaid-_r_tl_
[data-look="neo"].node polygon{stroke:url(#chatgpt-mermaid-_r_tl_-gradient);filter:drop-shadow(
1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_tl_ [data-look="neo"].swimlane.cluster
rect{filter:none;}#chatgpt-mermaid-_r_tl_ [data-look="neo"].node

path{stroke:url(#chatgpt-mermaid-_r_tl_-gradient);stroke-width:1px;}#chatgpt-mermaid-_r_tl_ [data-look="neo"].node .outer-path{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_tl_ [data-look="neo"].node .neo-line path{stroke:rgb(31, 78, 148);filter:none;}#chatgpt-mermaid-_r_tl_ [data-look="neo"].node circle{stroke:url(#chatgpt-mermaid-_r_tl_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_tl_ [data-look="neo"].node circle .state-start{fill:#000000;}#chatgpt-mermaid-_r_tl_ [data-look="neo"].icon-shape .icon{fill:url(#chatgpt-mermaid-_r_tl_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_tl_ [data-look="neo"].icon-shape .icon-neo path{stroke:url(#chatgpt-mermaid-_r_tl_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_tl_ :root{--mermaid-font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,Segoe UI,Helvetica,Apple Color Emoji,Arial,sans-serif,Segoe UI Emoji,Segoe UI Symbol;}USERSMISSING_PERSON_CASESSIGHTINGSAREA_SUBSCRIPTIONSNOTIFICATION_JOBSAUDIT_LOGSsubmitssubmitssubscribesreceivestriggersreceivesperforms

Important database decisions:

- Use UUIDs for internal IDs and random, non-sequential public case codes.
- Index case status, creation time, last-seen time, and area fields.
- Add spatial indexing with PostGIS only if you need efficient distance-based queries.
- Keep exact contact details and sensitive sighting information out of public case responses.
- Consider separate tables for contact details and media permissions if your project expands.

6. Cloud services and deployment

You do not need to use every cloud service available. Choose one primary cloud architecture and implement it properly.



Option A — AWS-based architecture

- React frontend: Vercel or S3 with CloudFront.
- FastAPI: a container service such as Amazon ECS with Fargate.
- Database: Amazon RDS for PostgreSQL.
- Images: Amazon S3, using short-lived authorized upload/download links where appropriate.
- Notifications: Amazon SQS plus a worker or Lambda function, with an email delivery provider.
- Monitoring: Amazon CloudWatch.

This is the most straightforward option if you want to explain an AWS-centered architecture in your presentation.



Option B — Simpler student deployment

- Frontend: Vercel.
- FastAPI backend: Render.
- Database: Neon PostgreSQL.
- Images: Cloudinary.
- Notifications: an email provider and a separate background worker if supported by your hosting plan.

This is often easier to implement if you prefer familiar tools and want to focus on completing the application. You can still demonstrate cloud storage, managed databases, background processing, and load testing.

My recommendation is Option B for the first working version, with an explicit notification queue and worker added for the cloud computing demonstration. If your course specifically requires hands-on AWS services, choose Option A instead.

Use one main database, one object-storage service, and one notification mechanism. Avoid paying for duplicate infrastructure just to make the architecture look complex.

Secure image uploads

Do not store uploaded photos directly inside the PostgreSQL database. Store the file in object storage and save its key in the case record.

For safety:

- Accept only necessary image types and impose a strict size limit.
- Validate actual file content, not just the filename or browser-supplied MIME type.
- Generate storage object names on the server.
- Keep unapproved case images private.
- Require authorization before issuing temporary access links.
- Remove or restrict media when a case is resolved or a valid removal request is approved.

These controls align with OWASP's file-upload security guidance.



OWASP Cheat Sheet Series
+1

7. API design

Organize the FastAPI backend around resources, with permissions enforced on the server.

Method	Endpoint	Purpose
POST	`/auth/register`	Create an account
POST	`/auth/login`	Authenticate

GET	`/cases`	List approved public cases
GET	`/cases/{case_code}`	View a public case
POST	`/cases`	Submit a case
PATCH	`/cases/{case_code}/review`	Approve or reject — moderator only
PATCH	`/cases/{case_code}/resolve`	Resolve a case — authorized user only
POST	`/cases/{case_code}/sightings`	Submit a private sighting
GET	`/me/sightings`	View one's own sightings
GET/POST/DELETE	`/me/subscriptions`	Manage area subscriptions
GET	`/me/notifications`	View notification history
GET	`/admin/audit-logs`	Review administrative activity

For moderator operations, enforce permissions in FastAPI dependencies. Hiding buttons in React is not security.

You should also implement pagination, request validation, rate limits on sensitive endpoints, and consistent error responses.

8. Step-by-step development roadmap

I recommend building this in eight medium-sized milestones, with a GitHub Issue and feature branch for each milestone. This keeps the project manageable and makes it easier to track progress.

Week 1

Milestone 1 — Project foundation

Issue #1

- Create the React/Vite frontend and FastAPI backend.
- Configure PostgreSQL, environment variables, and database migrations.
- Define API error handling, code structure, and Git workflow.

- Create initial responsive pages and navigation.

Acceptance criteria: Frontend and backend run locally; the API connects to PostgreSQL; secrets are not committed to Git.

Week 2

Milestone 2 — Authentication and permissions

Issue #2

- Implement registration, login, logout, and password hashing.
- Add user, moderator, and administrator roles.
- Protect private endpoints and enforce ownership rules.
- Test unauthorized access and invalid credentials.

Acceptance criteria: Users cannot access moderator functions or other users' private information.

Week 3

Milestone 3 — Missing-person case management

Issue #3

- Build case submission and validation.
- Implement case status transitions.
- Add moderator review and duplicate-report checks.
- Connect case forms to the API and database.

Acceptance criteria: A new case remains private until approved, and every status transition is validated.

Week 4

Milestone 4 — Public directory and map

Issue #4

- Add search, filters, pagination, and case details.
- Display approved cases on a Leaflet map.
- Show approximate public locations.
- Add empty, loading, and error states.

Acceptance criteria: Users can discover approved cases without gaining access to private case or sighting data.

Week 5

Milestone 5 — Cloud image storage and sightings

Issue #5

- Configure S3 or Cloudinary for image storage.
- Implement safe upload and authorized retrieval.
- Add the private sighting submission workflow.
- Add moderation controls and an audit trail.

Acceptance criteria: Images are not accessible merely by guessing a URL, and sightings are never listed publicly.

Week 6

Milestone 6 — Area subscriptions and notifications

Issue #6

- Let users subscribe to selected areas.
- Create notification jobs when cases are approved.
- Process jobs with a queue and background worker.
- Record delivery attempts and implement safe retries.

Acceptance criteria: Matching subscribers receive alerts; failed deliveries can be retried without sending unnecessary duplicates.

Week 7

Milestone 7 — Cloud deployment and monitoring

Issue #7

- Deploy the frontend, backend, database, and storage.
- Configure HTTPS, environment secrets, and CORS.
- Add application logs, health checks, and error monitoring.
- Configure backups and test database restoration.

Acceptance criteria: The deployed application works end to end, and the team can demonstrate monitoring and recovery procedures.

Week 8

Milestone 8 — Testing, load experiment, and presentation

Issue #8

- Test authentication, permissions, case workflows, and notifications.
- Generate synthetic traffic using a load-testing tool.
- Measure latency, throughput, error rates, and notification delay.
- Fix bugs and prepare architecture diagrams, documentation, and demo data.

Acceptance criteria: The team can demonstrate a complete workflow and present actual performance measurements.

These are planning estimates for a moderate scope, not fixed deadlines. If your semester is shorter, prioritize the core workflows first and keep load testing and advanced monitoring as the final extension.

9. How to prove you used cloud computing

A cloud deployment alone may not demonstrate enough of the subject. Include a small experiment with measurable results.

Suggested experiment: asynchronous alert processing

Use synthetic accounts, fake cases, and test email recipients only.

Test A — Direct processing

Send notification requests synchronously from the API and measure response latency.

Test B — Queue and worker

Put notification jobs in a cloud queue and return the API response before delivery completes.

Compare these metrics

- API response time: median and 95th percentile.
- Notification delivery delay.
- Successful and failed requests.
- Queue backlog and retry count.
- Behavior when a worker is stopped and restarted.

Your report can include a table like this and fill it with measurements from your actual tests.

Metric	Direct approach	Queue-based approach
API response time	Measure	Measure

Notification completion time	Measure	Measure
Failed requests	Measure	Measure
Recovery after worker failure	Test	Test

Do not assume the queue-based approach will always be faster in every metric. Its main advantage is separating the user-facing request from background work and allowing queued jobs to be retried.

10. Estimated budget and cost control

For a student prototype, start with free tiers or low-cost services where available. Free-tier eligibility, limits, and pricing change, so verify them before deploying.

- Use one small backend instance and a managed PostgreSQL database.
- Store only a small set of sample images.
- Use a modest email quota and restrict test recipients.
- Set cloud budgets and usage alerts where available.
- Shut down paid resources that are not needed.
- Do not store real missing-person records during development.

The simplest architecture that supports your requirements is preferable to a large multi-service design you cannot afford or maintain.

11. Final deliverables

By the end of the project, aim to have:

- A working React application and FastAPI backend.
- A PostgreSQL database with documented relationships and migrations.
- Cloud image storage and area-based notification delivery.
- Authentication, moderator controls, private sightings, and audit logs.
- A deployed demonstration with safe synthetic data.
- Automated tests and a short load-testing report.
- A system architecture diagram, database ER diagram, API documentation, and final project report.

Useful references for your technical documentation include the AWS RDS and Lambda integration tutorial, the OWASP File Upload Cheat Sheet, and the OWASP Access Control guidance.